

I am working on the Math Flashcards Repair Lab project in this repository:

https://github.com/SETAPESU26/61_math_flashcards

Please analyze the existing code, especially `game/game_engine.py`, and identify the deliberate bug described in the README.

The bug is in `compute_expected_answer()`. It currently treats the two operands like strings instead of performing the actual mathematical operation.

Fix **ONLY** this bug first.

Requirements:

- Check `self.operator` explicitly.
- Support `+`, `-`, and `*`.
- Return the mathematically correct result for the selected operator.
- Do not add any new features yet.
- Do not change the project structure.
- Keep the existing coding style and behavior everywhere else.
- Explain what was wrong and what you changed.

After making the change, provide the updated code for the affected function and briefly explain how I can test it.

I analyzed the repository and the README. The deliberate bug is exactly in

`GameEngine.compute_expected_answer()`: the current implementation converts the two operands into a single string such as `"712"` and then converts that back to an integer, instead of applying the selected operator. The README explicitly identifies this as Task 1.  [GitHub +1](#)

Updated `compute_expected_answer()`

Replace **only this function** in `game/game_engine.py`:

`</>` Python

```
def compute_expected_answer(self):  
    # BUG FIX:  
    # Evaluate the operands using the selected mathematical operator.
```

```
if self.operator == "+":
    return self.num_a + self.num_b
elif self.operator == "-":
    return self.num_a - self.num_b
elif self.operator == "*":
    return self.num_a * self.num_b
```

What was wrong?

The original code was:

</> Python

```
def compute_expected_answer(self):
    # BUG SYMPTOM:
    # Operands are combined as strings instead of evaluated with mathematical operations.

    return int(f"{self.num_a}{self.num_b}")
```

For example, if:

</> Plain text

```
num_a = 7
num_b = 3
operator = "+"
```

the old code effectively did:

</> Python

```
int("73")
```

so the expected answer became **73** instead of **10**.

The fix explicitly checks `self.operator` and performs the appropriate arithmetic:

- `+` → `self.num_a + self.num_b`
- `-` → `self.num_a - self.num_b`
- `*` → `self.num_a * self.num_b`

No other functionality, structure, or features need to be changed. [GitHub](#)

How to test it

From the project root:

`</> Bash`

`python main.py`

Then test several cards:

Card	Correct answer
7 + 3	10
12 - 5	7
6 * 4	24

Enter the correct answer and press **Enter** or click **SUBMIT**. A correct answer should display **CORRECT!** and generate the next card. The existing submission flow already calls `compute_expected_answer()`, so no other code needs to be modified. [GitHub](#)

Only Task 1 is addressed here—the timer, streak multiplier, and division features from the README are intentionally left untouched. [GitHub](#)

Now implement Task 2 from the README: a per-question countdown timer.

Requirements:

- Add a 10-second timer for each question.
- Display a visible countdown timer bar below the question card.
- Display the remaining time clearly.
- Update the timer continuously in `game_engine.update()`.
- When the timer reaches zero:
 - Count the unanswered question as an incorrect attempt.
 - Display "TIME'S UP!".
 - Reset the consecutive streak and multiplier.
 - Automatically generate the next question.
 - Reset the timer for the new question.
- The timer must reset whenever a new question is generated after a correct answer or timeout.
- Keep the existing arithmetic bug fix unchanged.
- Do not modify unrelated files unless absolutely necessary.
- Do not add any features beyond the timer.
- Preserve the existing game structure and controls.

Please provide the exact code changes and explain how I can test the timer manually.

Give the code

Yes. I checked the current `README.md` and `game/game_engine.py`. Task 2 specifically requires a 10-second per-question countdown, a visible bar, updating in `update()`, and automatic timeout handling.  [GitHub](#)

One important point: **Task 3 (streak/multiplier) has not been implemented in the current repository**, so there are currently no `streak` or `multiplier` variables to reset. I will **not introduce Task 3 functionality**, because you asked to implement only the timer. The timeout will therefore reset all timer/question state while leaving the existing scoring behavior untouched. The existing arithmetic fix is also preserved.  [GitHub](#)

Exact changes to `game/game_engine.py`

1. Update `__init__()`

Add the timer state **before** `generate_new_card()`:

 Python

```
self.timer_limit = 10.0
self.time_remaining = self.timer_limit
self.last_update_time = pygame.time.get_ticks()

self.generate_new_card()
```

So the end of your `__init__()` should look like:

 Python

```
self.font_title = pygame.font.SysFont(None, 38)
self.font_hud = pygame.font.SysFont(None, 26)
self.font_card = pygame.font.SysFont(None, 56)
self.font_btn = pygame.font.SysFont(None, 24)

self.timer_limit = 10.0
self.time_remaining = self.timer_limit
self.last_update_time = pygame.time.get_ticks()

self.generate_new_card()
```

2. Update `generate_new_card()`

Reset the timer whenever a new question is generated.

</> Python

```
def generate_new_card(self):
    self.num_a = random.randint(3, 15)
    self.num_b = random.randint(2, 12)
    self.operator = random.choice(["+", "-", "*"])
    if self.operator == "-" and self.num_a < self.num_b:
        self.num_a, self.num_b = self.num_b, self.num_a

    self.input_box.clear()

    self.time_remaining = self.timer_limit
    self.last_update_time = pygame.time.get_ticks()
```

This means the timer resets after:

- starting the game
- a correct answer
- a timeout

3. Keep your Task 1 arithmetic fix unchanged

Your function should remain:

</> Python

```
def compute_expected_answer(self):
    if self.operator == "+":
        return self.num_a + self.num_b
    elif self.operator == "-":
        return self.num_a - self.num_b
    elif self.operator == "*":
        return self.num_a * self.num_b
```

Do **not** change this for Task 2.

4. Replace `update()`

The current repository has:

</> Python

```
def update(self):  
    pass
```

Replace it with:

</> Python

```
def update(self):  
    current_time = pygame.time.get_ticks()  
    elapsed_time = (current_time - self.last_update_time) / 1000.0  
    self.last_update_time = current_time  
  
    self.time_remaining -= elapsed_time  
  
    if self.time_remaining <= 0:  
        self.time_remaining = 0  
        self.total_attempts += 1  
        self.feedback_msg = "TIME'S UP!"  
        self.feedback_color = (240, 75, 75)  
        self.generate_new_card()
```

This makes the timer independent of the game's frame rate because it uses elapsed milliseconds from Pygame.

When it reaches zero:

1. The timer is set to `0`.
2. The question counts as an attempt.
3. `"TIME'S UP!"` is displayed.
4. A new question is generated.
5. The timer automatically resets to 10 seconds.

5. Add the timer bar in `render()`

Immediately after the question card rendering:

</> Python

```
card_str = f"{self.num_a} {self.operator} {self.num_b}"  
card_surf = self.font_card.render(card_str, True, (25, 30, 42))  
screen.blit(card_surf, (card_rect.centerx - card_surf.get_width() // 2, card_rect.centery -  
card_surf.get_height() // 2))
```

add:

</> Python

```
timer_bar_rect = pygame.Rect(self.width // 2 - 130, 212, 260, 14)
pygame.draw.rect(screen, (60, 65, 75), timer_bar_rect, border_radius=6)

timer_ratio = max(0, self.time_remaining / self.timer_limit)
timer_fill_rect = pygame.Rect(
    timer_bar_rect.x,
    timer_bar_rect.y,
    int(timer_bar_rect.width * timer_ratio),
    timer_bar_rect.height
)
pygame.draw.rect(screen, (80, 200, 110), timer_fill_rect, border_radius=6)

timer_text = self.font_hud.render(
    f"Time: {self.time_remaining:.1f}s",
    True,
    (245, 245, 245)
)
screen.blit(
    timer_text,
    (
```

This places the countdown **below the question card** and above the input box.

Complete updated `game_engine.py`

For convenience, here is the complete affected file with **only the Task 2 timer changes** plus your existing Task 1 arithmetic fix:

</> Python

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.score = 0
        self.total_attempts = 0
        self.feedback_msg = "Solve the card and press Enter!"
        self.feedback_color = (200, 205, 215)

        self.num_a = 0
```

```
self.num_b = 0
self.operator = "+"
```

```
box w, box h = 130, 44
```

Manual testing

Run:

</> Bash

```
python main.py
```

You should see:

</> Plain text

```
| 7 + 5 |
```



Time: 9.8s

Test 1 — Countdown

Don't type anything.

You should see:

</> Plain text

```
Time: 10.0s
```

```
Time: 9.9s
```

```
Time: 9.8s
```

```
...
```

```
Time: 0.1s
```

The bar should progressively shrink.

Test 2 — Timeout

Don't answer for 10 seconds.

Expected behavior:

</> Plain text

TIME'S UP!

The attempt count should increase:

</> Plain text

Score: 0 / 1

Then a new **arithmetic question** should automatically appear with:

</> Plain text

Time: 10.0s

Test 3 — Correct answer resets timer

If you get a question such as:

</> Plain text

7 + 5

and answer:

</> Plain text

12

the next question should immediately appear and its timer should start again at approximately:

</> Plain text

Time: 10.0s

Test 4 — Wrong answer

Enter an incorrect answer before the timer expires.

The existing behavior should remain unchanged:

</> Plain text

WRONG! Expected 12.

The timer should **continue running** because an incorrect submission does not generate a new question in the existing game behavior.

Test 5 — Arithmetic bug remains fixed

Check all three operations:

</> Plain text

7 + 5 → 12

10 - 4 → 6

6 * 3 → 18

The Task 1 fix remains untouched.  GitHub

Files changed: only `game/game_engine.py`. No new files or project-structure changes are required.

Now implement Task 3 from the README: consecutive correct-answer streak multipliers.

Requirements:

- Track the number of consecutive correct answers.
- Start the streak at 0.
- Award the normal 1x score initially.
- Give a 2x multiplier when the player reaches 3 consecutive correct answers.
- Give a 3x multiplier when the player reaches 5 consecutive correct answers.
- Add the multiplied points to the existing score.
- Display the current streak and multiplier in the game interface.
- Reset the streak and multiplier to 1x after any incorrect answer.
- Reset the streak and multiplier to 1x after a timeout.
- Keep the existing timer and arithmetic functionality working.
- Do not remove or redesign existing functionality.
- Do not add unrelated features.

Please implement this cleanly in `game_engine.py` and explain the scoring logic with a small example such as five consecutive correct answers.

Implement Task 3 in `game/game_engine.py` by adding only the streak/multiplier state, updating scoring, resetting on wrong/timeout, and displaying the values. The existing Task 1 arithmetic fix and Task 2 timer remain unchanged.

1. Add streak state in `__init__()`

Place this after `self.total_attempts = 0`:

`</>` Python

```
self.streak = 0
self.multiplier = 1
```

So:

`</>` Python

```
self.score = 0
self.total_attempts = 0
self.streak = 0
self.multiplier = 1
self.feedback_msg = "Solve the card and press Enter!"
self.feedback_color = (200, 205, 215)
```

2. Update `submit_answer()`

Replace the existing `submit_answer()` with:

`</>` Python

```
def submit_answer(self):
    val_str = self.input_box.text.strip()

    if not val_str or val_str == "-":
        self.feedback_msg = "Type an answer first!"
        self.feedback_color = (240, 175, 40)
        return

    user_answer = int(val_str)
    expected = self.compute_expected_answer()
    self.total_attempts += 1

    if user_answer == expected:
        self.streak += 1

        if self.streak >= 5:
            self.multiplier = 3
        elif self.streak >= 3:
            self.multiplier = 2
        else:
```

3. Update timeout handling

Your existing timer code:

</> Python

```
if self.time_remaining <= 0:
    self.time_remaining = 0
    self.total_attempts += 1
    self.feedback_msg = "TIME'S UP!"
    self.feedback_color = (240, 75, 75)
    self.generate_new_card()
```

must become:

</> Python

```
if self.time_remaining <= 0:
    self.time_remaining = 0
    self.total_attempts += 1

    self.streak = 0
    self.multiplier = 1

    self.feedback_msg = "TIME'S UP!"
    self.feedback_color = (240, 75, 75)

    self.generate_new_card()
```

This preserves the Task 2 timeout behavior while resetting the Task 3 streak.

4. Display streak and multiplier

In `render()`, after the score display:

</> Python

```
score_surf = self.font_hud.render(
    f"Score: {self.score} / {self.total_attempts}",
    True,
    (255, 220, 80)
)
screen.blit(
    score_surf,
    (self.width // 2 - score_surf.get_width() // 2, 58)
)
```

add:

</> Python

```
streak_surf = self.font_hud.render(
    f"Streak: {self.streak}    Multiplier: {self.multiplier}x",
    True,
    (255, 220, 80)
)
screen.blit(
    streak_surf,
    (
        self.width // 2 - streak_surf.get_width() // 2,
        78
    )
)
```

The rest of your Task 2 timer rendering remains unchanged.

Complete relevant `game_engine.py`

Here is the complete version incorporating Task 1 + Task 2 + Task 3, without adding unrelated functionality:

</> Python

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.score = 0
        self.total_attempts = 0

        # Task 3: streak and multiplier
        self.streak = 0
        self.multiplier = 1

        self.feedback_msg = "Solve the card and press Enter!"
        self.feedback_color = (200, 205, 215)
```

Scoring logic

The important rule is:

</> Plain text

Streak 0-2 → 1x
Streak 3-4 → 2x
Streak 5+ → 3x

For example, starting with `Score = 0` :

Correct answer	Streak	Multiplier	Points earned	Total score
1st	1	1x	+1	1
2nd	2	1x	+1	2
3rd	3	2x	+2	4
4th	4	2x	+2	6
5th	5	3x	+3	9

So five consecutive correct answers give 9 total points.

If the player then answers incorrectly:

</> Plain text

Streak → 0
Multiplier → 1x

The next correct answer therefore earns only 1 point.

If the timer expires, exactly the same reset occurs:

</> Plain text

TIME'S UP!
Streak → 0
Multiplier → 1x

The existing 10-second timer and Task 1 arithmetic implementation remain intact.

Now implement Task 4 from the README: integer division support.

Requirements:

- Add "/" to the randomly selected operators in `generate_new_card()`.
- Division questions must always produce a whole-number answer with no remainder.
- Generate the divisor and quotient first, then calculate the dividend so the division is exact.
- Update `compute_expected_answer()` to correctly handle division using integer division.
- Keep the existing `+`, `-`, and `*` behavior unchanged.
- Preserve the subtraction safety that prevents negative results.
- Make sure the timer still works for division questions.
- Make sure the streak and multiplier system still works for division questions.
- Do not add any unrelated features.
- Keep the existing project structure.

After implementing it, review the complete `game_engine.py` for obvious integration problems and explain how I can test all four tasks manually.

Yes. For Task 4, the cleanest approach is to change only the question-generation logic and expected-answer logic while preserving the Task 1–3 behavior.

The key detail is that for `/`, we should generate:

`</>` Plain text

```
divisor * quotient = dividend
```

rather than generating two arbitrary numbers. That guarantees an exact integer answer.

Updated `game/game_engine.py`

`</>` Python

```
import random
import pygame
from game.text_box import TextBox
```

```
class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.score = 0
        self.total_attempts = 0

        # Task 3: streak and multiplier
        self.streak = 0
        self.multiplier = 1

        self.feedback_msg = "Solve the card and press Enter!"
        self.feedback_color = (200, 205, 215)
```

What changed for Task 4?

1. / was added

Previously:

</> Python

```
self.operator = random.choice(["+", "-", "*"])
```

Now:

</> Python

```
self.operator = random.choice(["+", "-", "*", "/"])
```

2. Division is generated differently

For division:

</> Python

```
self.num_b = random.randint(2, 12)
quotient = random.randint(2, 12)
self.num_a = self.num_b * quotient
```

For example:

</> Plain text

```
divisor  = 7
quotient = 4

dividend = 7 × 4
          = 28

Question = 28 / 7
Answer   = 4
```

Therefore, you will never get something like:

</> Plain text

```
10 / 3
```

where the answer has a remainder.

3. Integer division was added

</> Python

```
elif self.operator == "/":
    return self.num_a // self.num_b
```

Because generation guarantees exact division, `//` produces the intended whole-number answer.

4. Existing operations remain unchanged

</> Plain text

```
+ → addition
- → subtraction
* → multiplication
/ → exact integer division
```

The subtraction safety is still present:

</> Python

```
if self.operator == "-" and self.num_a < self.num_b:
    self.num_a, self.num_b = self.num_b, self.num_a
```

Integration review

The four tasks now work together as follows:

Task	Functionality	Integration
Task 1	<code>+</code> , <code>-</code> , <code>*</code> arithmetic	Preserved
Task 2	10-second countdown	Works for <code>/</code> too
Task 3	Streak + 2x/3x multiplier	Works for <code>/</code> too
Task 4	Exact integer division	Added

There is no separate division-specific timer or scoring logic—the existing systems automatically apply to division questions because `generate_new_card()` and `compute_expected_answer()` handle the operator consistently.

Manual testing of all 4 tasks

Run:

```
</> Bash
```

```
python main.py
```

Test Task 1 — Arithmetic

Test questions such as:

```
</> Plain text
```

```
7 + 4 = 11
```

```
12 - 5 = 7
```

```
6 * 3 = 18
```

Correct answers should increase the score.

Test Task 2 — Timer

Don't answer a question.

You should see:

```
</> Plain text
```

```
Time: 10.0s
Time: 9.9s
Time: 9.8s
...
Time: 0.0s
```

Then:

`</>` Plain text

```
TIME'S UP!
```

A new question should appear with the timer reset to approximately `10.0s`.

Test Task 3 — Streak

Answer five questions correctly in a row.

You should see:

`</>` Plain text

```
1st correct → Streak: 1   Multiplier: 1x
2nd correct → Streak: 2   Multiplier: 1x
3rd correct → Streak: 3   Multiplier: 2x
4th correct → Streak: 4   Multiplier: 2x
5th correct → Streak: 5   Multiplier: 3x
```

Score progression:

`</>` Plain text

```
+1
+1
+2
+2
+3
----
9 total
```

Then deliberately enter a wrong answer.

It should reset to:

</> Plain text

Streak: 0 Multiplier: 1x

Letting the timer expire should produce the same reset.

Test Task 4 — Division

Keep playing until / appears.

For example:

</> Plain text

28 / 7

Enter:

</> Plain text

4

You should get:

</> Plain text

CORRECT!

and the streak should increase.

Try another:

</> Plain text

72 / 8

Answer:

</> Plain text

9

The score should receive the **current multiplier**, proving that division is integrated with Task 3.

Also verify that every division question has an exact answer—there should be **no remainder-based questions**.

Finally, let a division question time out. It should:

⌄ Plain text

```
TIME'S UP!  
Streak → 0  
Multiplier → 1x  
New division/arithmetic question → generated  
Timer → reset to 10 seconds
```

That gives you a complete Task 1–4 implementation without changing the project structure or introducing unrelated features.

Review the current implementation against every requirement in the README for this Math Flashcards Repair Lab.

Verify these four tasks individually:

1. Correct arithmetic calculation for +, -, and *
2. 10-second per-question countdown timer with timeout handling
3. Consecutive correct-answer streak with 2x at 3 correct answers and 3x at 5 correct answers
4. Integer division with no remainder

Also verify:

- Incorrect answers reset the streak and multiplier.
- Timeouts reset the streak and multiplier.
- Correct answers generate a new question.
- The timer resets for every new question.
- Empty submissions do not count as failed attempts.
- Existing controls still work.
- No unnecessary project files or features were added.

Do not introduce new features during this review.

Give me a concise checklist showing PASS/FAIL for each

requirement and identify any issues that should be fixed before I record the final 10-second demonstration video.

Worked for 10s

I reviewed the README requirements and also checked the current `main` branch of the repository. There is one important distinction: **the GitHub repository currently still contains the original unmodified `game_engine.py`**—its `compute_expected_answer()` is still the deliberate concatenation bug and `update()` is still `pass`.  GitHub +1

So the checklist below evaluates the **Task 1–4 implementation we built in this conversation**, assuming your local `game/game_engine.py` matches the complete version from the previous step.

Final verification checklist

Requirement	Status	Review
Task 1: <code>+</code> calculation	✓ PASS	Uses <code>self.num_a + self.num_b</code> .
Task 1: <code>-</code> calculation	✓ PASS	Uses <code>self.num_a - self.num_b</code> .
Task 1: <code>*</code> calculation	✓ PASS	Uses <code>self.num_a * self.num_b</code> .
Subtraction never produces negative result	✓ PASS	Operands are swapped during generation when necessary.
Task 2: 10-second timer	✓ PASS	<code>timer_limit = 10.0</code> .
Timer continuously updates in <code>update()</code>	✓ PASS	Uses <code>pygame.time.get_ticks()</code> and elapsed time.
Visible timer bar	⚠ FIX	Timer bar exists, but its text/input positioning overlaps. See below.
Remaining time displayed clearly	⚠ FIX	The timer text is positioned at <code>y=228</code> , while the input box starts at <code>y=230</code> .
Timeout registers failed attempt	✓ PASS	<code>total_attempts</code> increments on timeout.
Timeout displays <code>TIME'S UP!</code>	✓ PASS	Correct feedback message is set.
Timeout generates next question	✓ PASS	Calls <code>generate_new_card()</code> .
Timeout resets streak	✓ PASS	<code>self.streak = 0</code> .

Requirement	Status	Review
Timeout resets multiplier	✓ PASS	<code>self.multiplier = 1</code> .
Timer resets for new question	✓ PASS	<code>generate_new_card()</code> resets timer to 10 seconds.
Task 3: streak starts at 0	✓ PASS	<code>self.streak = 0</code> .
Initial multiplier is 1x	✓ PASS	<code>self.multiplier = 1</code> .
3 consecutive correct → 2x	✓ PASS	<code>streak >= 3</code> sets multiplier to 2.
5 consecutive correct → 3x	✓ PASS	<code>streak >= 5</code> sets multiplier to 3.
Multiplied points added to score	✓ PASS	<code>self.score += self.multiplier</code> .
Streak/multiplier displayed	✓ PASS	HUD displays both values.
Wrong answer resets streak	✓ PASS	Streak becomes 0.
Wrong answer resets multiplier	✓ PASS	Multiplier becomes 1x.
Correct answer generates new question	✓ PASS	Calls <code>generate_new_card()</code> .
Task 4: <code>/</code> operator generated	✓ PASS	<code>/</code> is included in operator selection.
Division has no remainder	✓ PASS	Divisor and quotient are generated first; dividend = divisor × quotient.
Integer division calculation	✓ PASS	Uses <code>self.num_a // self.num_b</code> .
Division works with timer	✓ PASS	Same timer mechanism applies to all operators.
Division works with streak/multiplier	✓ PASS	Correct division answers follow the same scoring path.
Empty submission doesn't count	✓ PASS	Return occurs before <code>total_attempts += 1</code> .
Enter key still works	✓ PASS	Existing <code>K_RETURN</code> handling preserved.
SUBMIT button still works	✓ PASS	Existing mouse handling preserved.

Requirement	Status	Review
No unrelated features added	✓ PASS	Only Tasks 1–4 functionality was added.
Project structure preserved	✓ PASS	Only <code>game_engine.py</code> needs modification.

The README itself confirms the four required tasks and the empty-submission behavior.  [GitHub](#)

⚠ One issue to fix before your final video

There is a **layout collision** in the version we built.

Currently:

⌕ Python

```
timer_text → y = 228
input_box  → y = 230
```

So the timer text can visually overlap the answer input box.

This doesn't break the underlying functionality, but it **does affect the required visible timer demonstration**.

Recommended small fix

Move the input controls slightly down:

⌕ Python

```
box_w, box_h = 130, 44
self.input_box = TextBox(width // 2 - 110, 255, box_w, box_h)
self.submit_btn = pygame.Rect(width // 2 + 30, 255, 90, box_h)
```

And move the feedback message down accordingly:

⌕ Python

```
screen.blit(
    msg_surf,
    (
        self.width // 2 - msg_surf.get_width() // 2,
        320
    )
)
```

This is **not a new feature or redesign**; it simply prevents the Task 2 timer from overlapping the existing controls.

Important: your GitHub repository

Right now, the repository URL you gave me still shows the original code:

Python

```
def compute_expected_answer(self):  
    return int(f"{self.num_a}{self.num_b}")
```

and:

Python

```
def update(self):  
    pass
```

So the GitHub version is **NOT** yet ready for your final demonstration.  GitHub

If your local version contains the Task 1–4 code we developed, then make the small layout correction above, run it, and **push that version to GitHub before submitting**.

Final video test sequence

For the 10-second final demonstration, make sure you visibly capture:

1. Normal arithmetic → correct answer.
2. Timer bar counting down.
3. Timeout → **TIME'S UP!** → new question.
4. Streak 1x → 2x at the 3rd correct answer.
5. 3x at the 5th correct answer.
6. Division question, e.g. **28 / 7 = 4**.
7. Wrong answer or timeout showing **streak reset to 0 / 1x**.

That will cover essentially every important requirement in the README's four tasks.  GitHub

Verdict: Functionally **PASS** for Tasks 1–4, but fix the timer/input overlap and ensure your completed **game_engine.py** is actually pushed to the GitHub repository before recording the final video.